#B Pass the JWT as the assertion parameter

The claims in the JWT are the same as those used for client authentication, with the following exceptions:

- The `sub` claim should be the username of the service account rather than the client ID.
- The `iss` claim may also be different from the client ID depending on how the AS is configured.

There is an important difference in the security properties of the two methods, and this is often reflected in how the AS is configured. When the client is using a JWT to authenticate itself, the JWT is a self-assertion of identity. If the authentication is successful, then the AS issues an access token authorized by the client itself. In the JWT bearer grant, the client is asserting that it is authorized to receive an access token on behalf of the given user, which may be a service account or a real user. Because the user is not present to consent to this authorization, the AS will usually enforce stronger security checks before issuing the access token. Otherwise a client could ask for access tokens for any user it liked, without the user being involved at all. For example, an AS might require separate registration of trusted JWT issuers with settings to limit which users and scopes they can authorize access tokens for.

An interesting aspect of JWT bearer authentication is that the issuer of the JWT and the client can be different parties. You'll make use of this capability in section 11.5.3 to harden the security of a service environment by ensuring that pods running in Kubernetes don't have direct access to privileged service credentials.